
Advanced Root Detection Implementation

Military Exam Application — Security Architecture

Prepared By:	Divergent Technologies Ltd
Document Version:	1.0
Last Updated:	July 2026
Classification:	CONFIDENTIAL / MILITARY GRADE

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Advanced Root Detection Package
4. SecurityService (RASP Layer)
5. Threat Categories and Detection
6. Blocking Scenarios
7. Pre-Exam Security Gate Flow
8. Login Screen Monitoring
9. Exam Watchdog (Runtime Protection)
10. Violation Handling
11. Deployment and Environment Modes
12. Release Hardening (ProGuard)
13. Key Source Files

1. Executive Summary

The Military Exam application implements a rigorous, multi-layered security framework engineered around Divergent Technologies Ltd's proprietary local `advanced_root_detection` Flutter plugin. By merging high-performance Kotlin/Java detectors, an isolated alternate Android process sandbox, and a low-level native NDK engine (`libshield.so`), the application provides definitive identification of platform threats including runtime rooting, function-hooking systems, active debuggers, hypervisors, binary tampering, unauthorized device builds, and environmental manipulation.

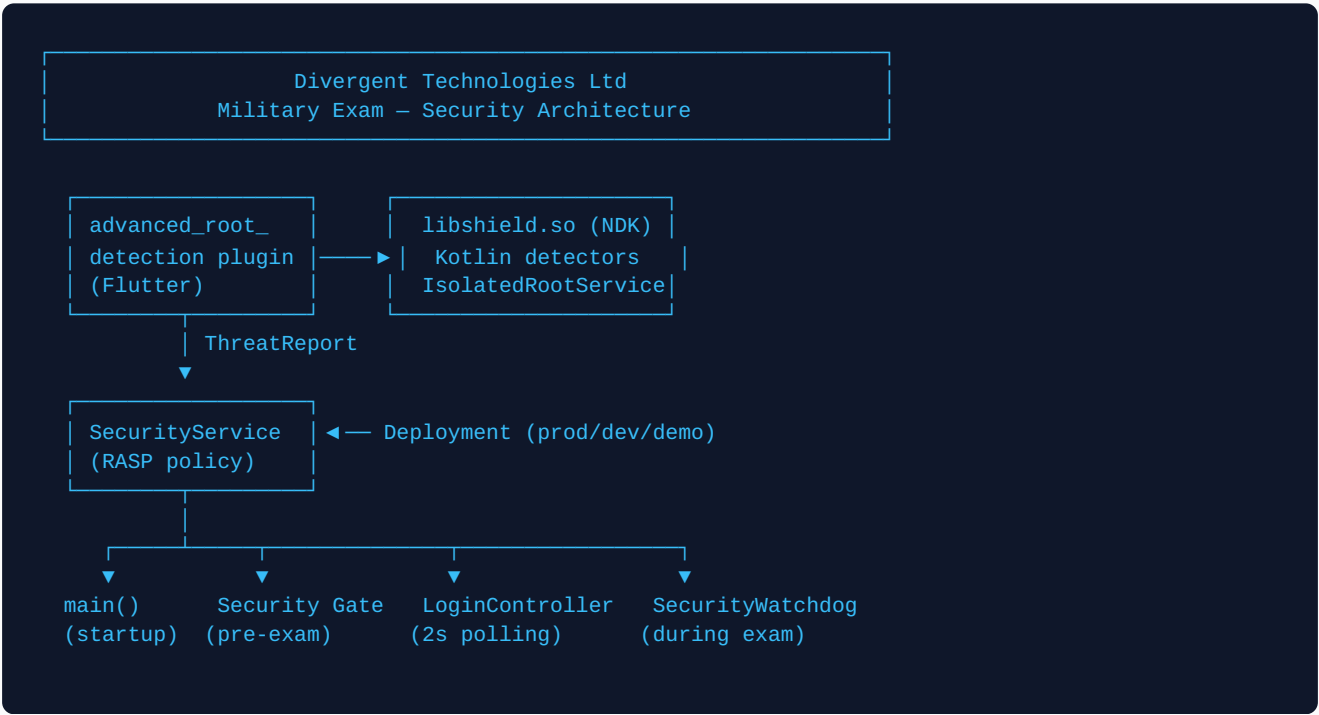
This runtime security mechanism is wrapped within a central `SecurityService` orchestrator, translating low-level system threats into environment-aware policy constraints across four specific application lifecycles:

Lifecycle Stage	Enforcement Component	Security Objective
Cold Start	<code>main()</code> + <code>SecurityService.initialize()</code>	Executes hard termination prior to UI rendering if high-criticality threats exist.
Pre-Exam	Security Gate + contextual Sub-Gates	Validates base platform integrity, developer modes, physical configurations, and connectivity.
User Login	<code>LoginController</code> active polling loop	Maintains continuous system integrity checks while sensitive authentication inputs occur.
Active Exam	<code>SecurityWatchdogService</code> daemon	Protects operational lifecycle states, suppresses screen streaming, and forces system isolation.

Fail-Closed Paradigm Architecture: All internal platform interaction exceptions, missing channel communication links, or timeout exceptions are explicitly intercepted and evaluated as high-risk, non-compliant target integrity configurations, preventing execution fallback.

2. Architecture Overview

The layout below details the decoupled flow of information originating within the hardware-adjacent low-level native modules, bubble up through the Flutter plugin contract wrapper, interface through the policy validation engine, and ultimately target specific presentation and execution lifecycles.



3. Advanced Root Detection Package

Internal Framework Location: `packages/advanced_root_detection/`

Integration Strategy: Statically bound native integration resolved via direct local references in the core system's `pubspec.yaml` descriptor file.

Dart API Layer Definitions

Interface Signature	Functional Scope & Operational Profile
<code>performCheck(SecurityConfig)</code>	Triggers complete multi-layered deep diagnostic hardware scan. Synchronously aggregates native/managed metrics and formats a clean ThreatReport block.
<code>verifyBeforeSensitiveOp(SecurityConfig)</code>	Ultra-low latency, highly direct micro-check explicitly targeted for high-frequency runtime polling hooks and security-critical execution blocks.
<code>startMonitoring()</code> / <code>stopMonitoring()</code>	Initializes an unmanaged background streaming engine loop supplying continuous system profile details (reserved for future asynchronous integration).
<code>threatStream</code>	Public event bus broadcasting live platform posture alterations dynamically across the application runtime environment.

Native Infrastructure Mapping

To bypass runtime intercept frameworks and anti-root prevention wrappers (such as Magisk DenyList), security checks execute dynamically out-of-process via an alternate memory execution vector:

- **Plugin Layer (`AdvanceRootDetectionPlugin.kt`)**: Serves as the high-level Kotlin interface mapping incoming platform calls from the Dart `MethodChannel` infrastructure down to standard native arrays.
- **Isolated Runtime Sandbox (`IsolatedRootService.kt`)**: Instantiated as an autonomous Android sub-process leveraging specialized OS isolation parameters. It performs validation tasks out-of-process, guaranteeing clean analysis metrics independent of memory hooks target-bound to the main package identifier.
- **Kotlin Detection Suite (`detectors/`)**: Modulated codebase components analyzing rooting frameworks, hook hooks, emulator targets, active debugger extensions, system properties adjustments, and digital signature tampering.
- **Native Core Layer (`cpp/` → `libshield.so`)**: Low-level compiled asset performing raw Linux systems querying, direct memory segment scanning, and hardware verification bindings.

Native C++ Execution Capabilities (`libshield.so`)

The core NDK compilation profile exercises low-level kernel interfaces directly to ensure structural accuracy:

- Reads `TracerPid` parameters and executes proactive `ptrace` operational loop locks to stop active process debugging attachments (`anti_debug.cpp`).
- Performs raw textual file parsing of internal `/proc/self/maps` records to immediately pinpoint invalid memory mappings, hook scripts, or active Frida framework injections (`hook_detector.cpp`).
- Scans dynamic runtime metadata tables utilizing `dl_iterate_phdr` hooks to identify hidden loaded libraries linked with Zygisk or Magisk extensions (`zygisk_detector.cpp`).
- Validates physical memory addresses and checks code section write access blocks across the running execution context (`integrity_check.cpp`).
- Detects overlay directory mounts and suspicious partition configurations via virtual file system checks (`statfs`).

4. SecurityService (RASP Layer)

Primary Operations Vector: `lib/core/services/security_service.dart`

The `SecurityService` framework behaves as a localized Runtime Application Self-Protection (RASP) decision module. It acts as a singleton proxy layer that normalizes raw threat records down to clear state maps evaluated against the running environment configuration (Development, Staging, or Production).

Policy Validation Matrices

Threat handling rules apply granular actions based on the specific classification of the detected security event:

Structural Event Classification	System Status Flag Integration	Assigned Policy Routing
Package Integrity Defect	<code>isIntegrityViolated</code>	Immediate Hard Stop
Privileged Access / Escalation	<code>isRooted</code> / <code>isJailbroken</code>	Immediate Hard Stop
Runtime Instrumentation Hook	<code>isHooked</code>	Immediate Hard Stop
Debugger Attachment Detected	<code>isDebuggerAttached</code>	Immediate Hard Stop
Virtual Hardware Target	<code>isEmulator</code>	Conditional Block (Production Only)
Developer Configuration Active	Custom Heuristic (USB Debugging / ADB)	Redirect to Remediation

5. Threat Categories and Detection

System metrics map directly down to a standardized taxonomy defining specific exposure fields. Every operational threat parsed out of lower system domains assigns clear severity indexes, used to evaluate execution blocks dynamically:

- **privilegedAccess [High to Critical]:** Triggered by matching binaries (e.g., `su`), active management utility packages (Magisk), exposed kernel nodes, or active write permissions assigned to primary read-only directories.
- **runtimeManipulation [Critical]:** Triggered when checking dynamic linkages, dynamic system libraries containing trace instructions, unexpected function hooks, or inline overrides targeting standard system functions.
- **debuggerAttached [High]:** Assessed via low-level system execution values, tracing states via internal files, or system developer parameters.
- **integrityViolation [Critical]:** Occurs if digital application signatures break, resource configurations match tampering templates, or modifications are found within core application binaries.

6. Blocking Scenarios

The application security policy enforces clear operational pathways whenever environment validation checks encounter deviations from authorized baselines:

ID	Triggering Condition	Enforcement Context	Execution Action	Target Location
1	Critical RASP Detection	Startup Initialization	Immediate Termination	<code>DeviceCompromisedApp</code>
2	Root / Hook Injection	Pre-Exam Security Gate	Disable Next Actions	Remain on <code>/security/gate</code>
3	Developer Mode Engaged	Gate or Login Evaluation	Navigation Override	<code>/security/developer-mode</code>
4	Airplane Mode Inactive	Gate or Login Evaluation	Navigation Override	<code>/security/airplane-mode</code>
5	Local Network Disconnection	Gate or Login Evaluation	Navigation Override	<code>/security/wifi-mode</code>
6	Camera Permission Denied	Post-Gate Validation	Navigation Override	<code>/security/camera-permission</code>
7	Airplane Mode Deactivated	Active Testing Session	Lock and Submit	<code>/violation</code>
8	Application Backgrounded	Active Testing Session	1s Grace → Lock	<code>/violation</code>
9	RASP Subsystem Timeout	Active Testing Session	Lock and Submit	<code>/violation</code>
10	Network State Instability	Active Testing Session	Display UI Warning	No Interruption

7. Pre-Exam Security Gate Flow

Primary Controller Context: `SecurityGateController`

Navigation Parameter Location: `/security/gate`

Prior to accessing candidate authentication screens, the application forces sequentially structured checks to confirm environmental compliance. Progression past the initial security gate requires fulfilling a comprehensive 8-point checklist:

- 1. Platform Security Validation:** Invokes `CheckDeviceIntegrityUseCase` to fetch fresh system health snapshots from the RASP native suite. The presence of core rooting mechanisms, invalid signatures, hardware emulation, or system identity spoofing triggers an immediate system lockout.
- 2. Hardware Radio Validation:** The device hardware configuration must demonstrate that cellular radios are offline by validating that Airplane Mode is active.

3. Localized Network Infrastructure Check: Confirms that wireless networking adapters remain active and successfully bound to the specific testing network topology despite radio isolation configurations.

8. Login Screen Monitoring

Controller Logic Context: `LoginController`

User Presentation Interface: `login_page.dart`

To block threat vectors introduced after clearing initial system checks but before authenticating testing profiles, the login environment executes constant system posture analysis routines every 2 seconds via configuration parameters mapped directly to `AppConstants.airplaneModePollInterval`.

9. Exam Watchdog (Runtime Protection)

Operational Service Daemon: `SecurityWatchdogService`

Initialization Vector: `StartSecurityWatchdogUseCase`

Securing the candidate environment during active assessment conditions relies on aggressive polling infrastructure and low-level event interception routines tailored for sensitive phases.

Active System Metrics Policy

Policy Configuration Variable	Target Baseline Value	Operational Enforcement Profile
<code>requireAirplaneMode</code>	<code>true</code>	Forces absolute radio isolation throughout testing window.
<code>monitorLifecycle</code>	<code>true</code>	Intercepts minimization, window partitioning, or focus changes.
<code>preventScreenCapture</code>	<code>true</code>	Hardware-level pixel masking utilizing the operating system's window flags.
<code>pollInterval</code>	3 Seconds	High-frequency background checking loop cadence.

10. Violation Handling

Processing Engine Pipeline: `HandleSecurityViolationUseCase`

Upon detection of platform compliance deviations or intentional security breaches, the application switches to a locked state via coordinated local and remote tasks including lock activation, remote threat logging, and automatic submission pipelines.

11. Deployment and Environment Modes

Security rules dynamically scale based on the build target to maintain flexibility during QA and staging, while maintaining a strict lock down in production environments:

Operational Policy Variable	Development	Staging	Production	Demo Profile
<code>allowSideload</code>	<code>true</code>	<code>true</code>	<code>false</code>	<code>true</code>
<code>strictExamIntegrity</code>	<code>true</code>	<code>true</code>	<code>true</code>	<code>true</code>
<code>blockEmulator</code>	<code>false</code>	<code>false</code>	<code>true</code>	<code>false</code>

Critical Deployment Warning: The current target entry execution logic (`main.dart`) resolves environment states via temporary mock parameters (`Deployment.init(demo: true)`). Transitioning to operational readiness requires removing all temporary parameters.

12. Release Hardening (ProGuard)

To maintain code obfuscation metrics without dropping dynamic platform method connections, release builds explicitly protect critical package pathways from dead-code elimination routines within the R8 compiler across the engine bindings and dynamic JNI interfaces.

13. Key Source Files

Core Security Domain	Target Component Repository Path Location
Central RASP Singleton	<code>lib/core/services/security_service.dart</code>
Exam Session Guard Daemon	<code>lib/core/services/security_watchdog_service.dart</code>
Main Security Verification Gate	<code>lib/features/security_gate/presentation/controllers/security_gate_controller.dart</code>
Login Continuous Monitoring Polling Loop	<code>lib/features/auth/presentation/controllers/login_controller.dart</code>